
Interactive Terminal-Based Word Search Puzzle Generator in C

Course Code & Section: CSE115L.4 Project
Group No.: Group 02
Date of Submission: August 2026

Group Members

- **Md. Fahad Kamal Lingkon** (ID: 2621146042) — *Grid Generation, Algorithmic Engine & File I/O*
- **Md. Farhad Prodhan Fahim** (ID: 2624625642) — *UI/UX, Console Rendering & State Structures*
- **Mahiraj Promo** (ID: 2622117042) — *Search Engine, Menus & Input Validation*
- **Md. Mahedi Hassan Sumonto** (ID: 2624122042) — *Quality Assurance, Testing & Bug Fixing*

1 Group Contribution Table

Table 1: Module Allocation and Contribution Breakdown

Member Name	Student ID	Assigned Modules / Responsibilities	Contribution
Md. Fahad Kamal Lingkon	2621146042	Grid Generation, Algorithms & File I/O: Implemented vector mathematics (DR, DC), collision detection (canPlace), Monte Carlo randomized placement, and external file parsing (loadWordBankFromFile).	35.0%
Md. Farhad Prodhan Fahim	2624625642	UI/UX, Console Rendering & State Structures: Developed Win32 terminal color mapping (PAL), dynamic board rendering, checklist formatting, and designed the PlayerProfile struct for session tracking.	25.0%
Mahiraj Promo	2622117042	Search Engine, Menus & Input Validation: Built the 4-level nested matrix scan (findInGrid), hint candidate pool stack, string sanitization pipelines, and interactive game state loops.	20.0%
Md. Mahedi Hassan Sumonto	2624122042	Quality Assurance, Testing & Bug Fixing: Conducted full system integration testing, memory handling, stream buffer sanitization verification, edge-case debugging, and game loop stabilization.	20.0%

2 Objectives

- **Procedural Matrix Generation:** Design and construct a dynamic 15×15 grid capable of embedding vocabulary strings across all 8 cardinal and diagonal directions without hardcoded pathing.
- **Collision-Resistant Placement:** Implement mathematical vector models that validate grid boundaries and permit interlocking character intersections while rejecting conflicting letter overwrites.
- **Dynamic Resource Loading:** Implement standard File I/O mechanisms to parse external text files (.txt) for word banks, preventing hardcoded limitations and enhancing replayability.
- **Persistent State Tracking:** Utilize C structures (struct) to encapsulate and track player session data, including usernames, cumulative scores, and total puzzles completed.
- **Exhaustive Matrix Traversal:** Develop a multidimensional search engine to scan and verify user-spotted words in real time.
- **Defensive Input Handling:** Build robust input-sanitization pipelines preventing buffer overflow vulnerabilities, case discrepancies, and stream synchronization issues.

3 Development Tools & Header Files Justification

IDE & Compiler: Code::Blocks IDE using the GNU GCC Compiler (C99/C11 standard).

Required Header Files & Technical Justification

- **<stdio.h>(StandardInput/Output):** Essential for stream operations. Used for formatted console rendering (printf()), robust stream reading (fgets(), scanf()), and crucially for File I/O via FILE pointers (fopen(), fprintf(), fclose()) to read and generate the external word dictionary.
- **<stdlib.h> (Standard Utility Library):** Supplies the pseudo-random generation pipeline (rand(), srand()) and terminal control commands (system("cls")).
- **<string.h>(String&MemoryManipulation):** Provides low-level memory clearing (memset()), string length validation (strlen()), dictionary matching (strcmp()), and memory-safe copying (strncpy()).
- **<time.h> (Time Utilities):** Supplies time(NULL) to retrieve the Unix timestamp, seeding srand() so every game generates a unique matrix layout.
- **<ctype.h> (Character Handling):** Used for string normalization via toupper() and custom word sanitization via isalpha().
- **<windows.h> (Win32 Console API):** Conditionally compiled (#ifdef _WIN32) to acquire terminal output handles and control text attributes for vibrant color highlights.

4 Technical Writing (Logics, Screenshots & Explanations)

4.1 System Headers, Win32 API Wrappers & State Structures

As shown in **Screenshot 1** and **Screenshot 2**, the application begins by importing core headers and wrapping platform-specific Win32 console API calls to enable colored terminal output.

[Screenshot 1 & 2: System Setup, Constants & Profile Struct]

Displays standard header inclusions, conditional Win32 API declarations, directional offset vectors (DR, DC), and the PlayerProfile structure.

Figure 1: System inclusions, Win32 API initialization, macro definitions, and state encapsulation.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <time.h>
#include <ctype.h>

#ifdef _WIN32 #include
<windows.h> static HANDLE
hCon;
static void initConsole(void) { hCon = GetStdHandle(STD_OUTPUT_HANDLE); } static void setColor(int c) {
SetConsoleTextAttribute(hCon, (WORD)c); } static void resetColor(void) { SetConsoleTextAttribute(hCon, 7);
} static void clrscr(void) { system("cls"); }
#else
static void initConsole(void) {} static void setColor(int c) {
(void)c; }
static void resetColor(void) {} static void clrscr(void) {
system("clear"); } #endif

#define GS 15 /* Grid Size */
#define MAX_W 12 /* Maximum words per puzzle */
#define MAX_L 14 /* Maximum word length */
#define TRIES 1000 /* Monte Carlo placement attempts */
#define NDIRS 8 /* 8 Direction vectors */

/* Direction Offset Vectors */
static const int DR[NDIRS] = { 0, 0, 1, -1, 1, 1, -1, -1 }; static const int DC[NDIRS] = { 1, -1, 0, 0, 1, -1,
1, -1 };

/* Player Profile Structure */ typedef struct
{ char username[32];
int totalSessionScore; /* Total words found across all puzzles played */ int puzzlesCompleted;
} PlayerProfile;

static PlayerProfile player; /* Global player profile instance */
```

Listing 1: Win32 API Initialization and Global State Struct

1
2
3
4
5
6
7
8
9

10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38

Technical Explanation: The code utilizes conditional compilation (`#ifdef _WIN32`) to safely initialize `GetStdHandle()` and invoke `SetConsoleTextAttribute()` for Win32 environments, providing terminal color highlights without breaking cross-platform compilation. The matrix operations rely on coordinate offset arrays `DR` and `DC` to traverse 8 directions (0° to 315°). Session tracking is neatly encapsulated inside the `PlayerProfile` structure.

4.2 Dynamic File I/O Engine & Word Bank Sanitization

Screenshot 3 and **Screenshot 4** highlight the dynamic dictionary loading mechanism through `loadWordBankFromFile()`.

[Screenshot 3 & 4: File I/O Engine & Sanitization Pipeline]

Illustrates the failsafe file creation process, line-by-line reading via `fgets()`, carriage-return stripping, and uppercase normalization.

Figure 2: External file dictionary parser with automatic fallback generator and duplicate filter.

Listing 2: Dynamic External File Parsing and String Cleaning

```

static void loadWordBankFromFile(void) { FILE *fp =
    fopen("words.txt", "r"); if (!fp) { fp =
    fopen("words.txt", "w"); if (fp) {
        fprintf(fp, "ALGORITHM\nCOMPILER\nDATABASE\nNETWORK\nPYTHON\nBINARY\nSYNTAX\n");
        fprintf(fp, "FUNCTION\nPOINTER\nSTRUCT\nMEMORY\nKERNEL\nSERVER\nROUTER\nCACHE\n");
        fprintf(fp, "STACK\nQUEUE\nGRAPH\nARRAY\nLOOP\nCLASS\nOBJECT\nMODULE
            \nTHREAD\nPROCESS\n"); fclose(fp);
    }
    fp = fopen("words.txt", "r"); if (!fp) return; /*
    Failsafe */
}

char buf[128]; externalBankSz = 0;

while (fgets(buf, sizeof(buf), fp) && externalBankSz < MAX_BANK_SZ) { buf[strcspn(buf, "\r\n")] =
    '\0'; /* Strip newline characters */ toUpper(buf); /* Convert to uppercase */ int len =
    (int)strlen(buf); int duplicate = 0;
    for (int i = 0; i < externalBankSz; i++) { if (strcmp(externalBank[i],
        buf) == 0) { duplicate = 1;
            break;
        }
    }
    if (len >= 2 && len <= MAX_L && !duplicate) {

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23

24
25
26
27
28
29
30
31
32
33
34
35
36
37

```
        strncpy(externalBank[externalBankSz], buf, MAX_L);  
        externalBank[externalBankSz][MAX_L] = '\0'; externalBankSz++;  
    }  
    } fclose(fp);  
  
}
```

Technical Explanation: To avoid hardcoded vocabulary limits, loadWordBankFromFile() reads external words from words.txt. If the file is missing, a failsafe generates a default computer science dictionary on the fly. Each read string undergoes newline stripping via strcspn(), case conversion via toUpper(), length verification ($2 \leq \text{length} \leq 14$), and duplicate prevention using strcmp() before being cached in externalBank.

4.3 Algorithmic Logic: Monte Carlo Placement & Search Engine

Screenshot 5 and **Screenshot 6** showcase the procedural grid construction engine and the multidimensional search algorithm.

[Screenshot 5 & 6: Placement Logic & Search Traversal]

Shows tryPlaceWord() Monte Carlo loop, boundary checking in canPlace(), random ASCII noise generation in fillRandom(), and findInGrid() 4-nested matrix scan.

Figure 3: Monte Carlo word placement logic and $O(R \cdot C \cdot D \cdot L)$ matrix search engine.

Listing 3: Monte Carlo Word Placement and Matrix Search Engine

```

/* Validate placement along direction vector */ static int canPlace(const char *w, int
r0, int c0, int d) { int len = (int)strlen(w); for (int i = 0; i < len; i++) { int r = r0 + i * DR[d];
int c = c0 + i * DC[d];
    if (!inBounds(r, c)) return 0;
    if (grid[r][c] && grid[r][c] != w[i]) return 0; /* Collision check */
} return 1;
}

/* Monte Carlo placement loop */ static int
tryPlaceWord(int idx) { for (int t = 0; t < TRIES; t++) {
int d = rand() % NDIRS; int r0 = rand() % GS; int c0 =
rand() % GS;
    if (canPlace(W[idx], r0, c0, d)) { doPlace(W[idx], r0, c0, d, idx + 1);
        wRow[idx] = r0; wCol[idx] = c0; wDir[idx] = d; wPlaced[idx] = 1;
        nPlaced++; return 1;
    }
} return 0;
}

```

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26

```

}

/* Fill remaining unassigned grid cells with random uppercase noise */ static void fillRandom(void) { for
(int r = 0; r < GS; r++) for (int c = 0; c < GS; c++) if (!grid[r][c]) grid[r][c] = 'A' + rand() % 26;
}

/* Search engine: 4-nested loop matrix scanner */
static int findInGrid(const char *word, int *fr, int *fc, int *fd) { int len = (int)strlen(word);
    for (int r = 0; r < GS; r++) { for (int c = 0; c < GS; c++) { for (int
        d = 0; d < NDIRS; d++) { int ok = 1;
            for (int i = 0; i < len && ok; i++) { int nr = r + i * DR[d]; int nc = c + i * DC[d]; if
                (!inBounds(nr, nc) || grid[nr][nc] != word[i]) ok = 0;
            }
            if (ok) { *fr = r; *fc = c; *fd = d; return 1; }
        }
    }
} return 0;
}

```

27
 28
 29
 30
 31
 32
 33
 34
 35
 36
 37
 38
 39
 40
 41
 42
 43
 44
 45
 46
 47
 48
 49
 50
 51
 52
 53

Technical Explanation: The grid placement uses a Monte Carlo approach capping attempts at TRIES = 1000. It randomly picks origin (r_0, c_0) and direction d , placing words using doPlace() if canPlace() confirms zero

character conflicts. Empty cells are filled with random uppercase ASCII noise via `fillRandom()`. The real-time user verification engine `findInGrid()` executes a

4-nested search scan across all rows, columns, directions, and character indices in $O(R \cdot C \cdot D \cdot L)$ time.

4.4 Console UI Rendering & Word Checklist Display

As captured in **Screenshot 7** and **Screenshot 8**, the user interface renders the 15×15 grid with coordinate axes and prints an interactive word status list.

[Screenshot 7 & 8: Terminal Grid UI & Dynamic Word Checklist]

Displays color-coded cell rendering using color masks (fmask), dynamic player title headers, and menu action choices.

Figure 4: Dynamic grid renderer, color-coded word status checklist, and interactive menu.

```
static void printGrid(int showSol) { setColor(14); /* Bright Yellow  
    Headers */ printf("\n ");  
    for (int c = 0; c < GS; c++) printf("%3d", c + 1);  
    printf("\n                                     +-----+\n");
```

Listing 4: Grid Rendering, Word Checklist Formatting, and Actions Menu

1
2
3
4
5

```

for (int r = 0; r < GS; r++) {
    setColor(14); printf("%3d |", r + 1); for (int
    c = 0; c < GS; c++) { char ch = grid[r][c]; int
    fm = fmask[r][c];
        if (fm > 0) { setColor(PAL[(fm - 1) % 8]); /* Distinct color for found words */
            printf("[%c]", ch);
        } else {
            resetColor();
            printf(" %c ", ch);
        }
    }
    printf("\n");
}
resetColor();
}

static void printWordList(void) { setColor(11);
printf("\n --- %s'S PUZZLE --- FIND THESE %d WORDS (%d found) ---\n", player.username, nPlaced, nFound);
for (int i = 0; i < nW; i++) { if (!wPlaced[i])
    continue; if (wFound[i]) { setColor(PAL[i
    % 8]);
        printf(" (*) %-14s", W[i]); /* Marked found */
    } else {
        setColor(7);
        printf(" ( ) %-14s", W[i]); /* Pending word */
    }
    if ((i + 1) % 2 == 0) printf("\n");
}
printf("\n");
}

```

6
7
8
9
10
11
12
13
14

15
16
17
18
19
20
21
22
23
24
25

26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42

Technical Explanation: The grid renderer iterates through each cell, checking the mask matrix `fmask[r][c]`. If a cell belongs to a successfully identified word, it is highlighted using a color indexed from the palette array `PAL`. `printWordList()` displays remaining vs found words in realtime, personalized with the user's name from `player.username`.

4.5 Interactive Word Guessing, Win Detection & Dynamic Hint System

Screenshot9 and **Screenshot10** demonstrate the gameplay routines `doGuessWord()` and `doHint()`, handling player validation, completion tracking, and automated hints.

```
static void doGuessWord(void) { char
    buf[MAX_L + 10]; setColor(14);
    printf("\n Type the word you spotted: "); resetColor();

    if (!fgets(buf, sizeof(buf), stdin)) return;
```

Listing 5: Word Guessing Handler, Win State Tracking, and Hint Engine

1
2
3
4
5
6
7

[Screenshot 9 & 10: Win Detection & Dynamic Hint System]

Illustrates completion check triggering `player.puzzlesCompleted++`, along with the un-found word candidate pool stack in `doHint()`.

Figure 5: Word guessing validation pipeline, puzzle completion state updates, and dynamic hint generator.

```

if (strchr(buf, '\n') == NULL) flushInput(); buf[strcspn(buf, "\r\n")]
= '\0'; toUpper(buf); if (!buf[0]) return;

int widx = -1;
for (int i = 0; i < nW; i++) { if (wPlaced[i] && strcmp(W[i], buf) == 0) { widx = i; break; }
}

if (widx >= 0 && wFound[widx]) {
    setColor(14);
    printf(" \"%s\" has already been found!\n", buf); resetColor(); return;
}

int fr, fc, fd;
if (!findInGrid(buf, &fr, &fc, &fd)) { setColor(12); /* Red for invalid guess */
    printf(" \"%s\" is not in the grid. Check your spelling!\n", buf); resetColor(); return;
}

int colorId = (widx >= 0) ? widx + 1 : nPlaced + nFound + 1; markFoundCells(buf, fr, fc, fd, colorId);

if (widx >= 0) { wFound[widx] = 1; nFound++; player.totalSessionScore++; /* Increments
    cumulative score */

    if (nFound == nPlaced) { player.puzzlesCompleted++; /* Increment puzzle completion count */
        setColor(10);
        printf("\n ***** PUZZLE COMPLETE! All %d words found! *****\n", nPlaced);
        resetColor();
    }
}

}

static void doHint(void) { int pool[MAX_W], n = 0;
    for (int i = 0; i < nW; i++) {

```

8
9
10
11
12
13
14
15
16
17
18
19
20

21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44

45
46
47
48
49
50
51
52

```
        if (wPlaced[i] && !wFound[i]) pool[n++] = i;
    }
    if (n == 0) {
        setColor(10);
        printf(" All words found -- no hints needed!\n"); resetColor(); return;
    }
    int idx = pool[rand() % n]; setColor(14);
    printf(" HINT: \"%s\" starts at Row %d, Col %d (going %s)\n",
           W[idx], wRow[idx] + 1, wCol[idx] + 1, DNAME[wDir[idx]]); resetColor();
}
```

53
54
55
56
57
58
59
60
61
62
63
64

Technical Explanation: `doGuessWord()` checks user entries against placed words, updates cell masks upon verification, and evaluates puzzle completion ($n_{Found} == n_{Placed}$), triggering `player.puzzlesCom`. When players request assistance, `doHint()` dynamically constructs an index array pool of remaining hidden words and randomly reveals the start coordinate and vector orientation of one target word.

4.6 Custom Word Input Pipeline & Defensive Validation

Screenshot11 and **Screenshot12** show the interactive user word input module `getCustomWords()`.

[Screenshot 11 & 12: Custom Word Entry & Input Validation]

Source implementation of `getCustomWords()` featuring length checks, `isalpha()` alphabetic validation, and duplicate word detection.

Figure 6: Custom vocabulary entry pipeline with strict length, alphabetic, and duplicate checks.

```
static void getCustomWords(void) { int n;
    setColor(14);
    printf("\n How many words? (2-%d): ", MAX_W); resetColor();
    if (scanf("%d", &n) != 1) { flushInput(); n = 4; } flushInput();
    if (n < 2) n = 2; if (n > MAX_W) n =
    MAX_W;

    nW = 0;
    for (int i = 0; i < n; i++) { printf(" Word %d/%d (2-%d letters, letters only): ", i + 1, n, MAX_L); char buf[64];
        if (!fgets(buf, sizeof(buf), stdin)) { i--; continue; } if (strchr(buf, '\n') == NULL)
        flushInput(); buf[strcspn(buf, "\r\n")] = '\0'; toUpper(buf);

        int len = (int)strlen(buf);
```

Listing 6: Custom Word Entry Pipeline with Input Sanitization

13
14
15
16
17
18
19
20

```
if (len < 2 || len > MAX_L) {  
    setColor(12);  
    printf(" Length must be 2-%d -- skipped.\n", MAX_L); resetColor();  
    i--; continue;  
}  
  
int valid = 1;  
for (int k = 0; k < len; k++) {  
    if (!isalpha((unsigned char)buf[k])) { valid = 0; break; }  
}  
if (!valid) { setColor(12);  
    printf(" Letters only -- skipped.\n", MAX_L); resetColor();  
    i--; continue;  
}  
  
int duplicate = 0; for (int j = 0; j < nW; j++) { if (strcmp(W[j], buf) == 0) { duplicate =  
1; break; }  
}  
if (duplicate) { setColor(12);  
    printf(" Duplicate word -- skipped.\n"); resetColor();  
    i--; continue;  
}  
  
strncpy(W[nW], buf, MAX_L);  
W[nW][MAX_L] = '\0'; nW++;  
  
}  
}
```

21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37

38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54

TechnicalExplanation: When players choose to enter custom words, `getCustomWords()` prompts for vocabulary count and enforces strict sanitization. Inputs failing length requirements ($2 \leq \text{length} \leq 14$), containing non-alphabetic characters (`isalpha()`), or matching previously added entries (`strcmp()`) are rejected, ensuring clean grid generation without unexpected crash vectors.

4.7 Interactive Game Loop & Session Profile Summary

Screenshot 13, Screenshot 14, Screenshot 15, and Screenshot 16 capture the main game execution loop, choice switch handlers, and final session summary rendering in `main()`.

[Screenshot 13, 14, 15 & 16: Game Loop, Action Switch & Profile Summary]

Exhibits `main()` entry, profile initialization, game state menu loop, switch handlers for hints/regeneration, and final session struct output.

Figure 7: Main execution flow, state control menu loop, and final player profile summary output.

Listing 7: Main Program Loop and Session Summary Output


```

int main(void) { initConsole();
    srand((unsigned)time(NULL));

    clrscr(); printTitle();

    setColor(14);
    printf("\n Enter your Player Name: "); resetColor();
    if (fgets(player.username, sizeof(player.username), stdin)) { if (strchr(player.username, '\n')
        == NULL) flushInput(); player.username[strcspn(player.username, "\r\n")] = '\0';
    } if (player.username[0] == '\0') strcpy(player.username, "Guest");

    player.totalSessionScore = 0; player.puzzlesCompleted = 0;

    /* Main interactive loop */ int running =
    1, showSol = 0; while (running) { clrscr();
    printTitle(); printGrid(showSol);
    printWordList(); printMenu();

        int ch;
        if (scanf("%d", &ch) != 1) { flushInput(); continue; } flushInput();

        switch (ch) { case 1: doGuessWord(); showSol = 0; break;
            case 2: showSol = 1; printf("\n Solution shown!\n"); break; case 3: doHint(); break;
            case 4: generatePuzzle(); showSol = 0; break;
            case 5: pickFromBank(8); generatePuzzle(); showSol = 0; break; case 0: running = 0; break;
            default: printf(" Invalid choice (0-5).\n"); break;
        }
        printf(" Press Enter to continue..."); flushInput();
    }

    /* Print session summary from player struct */ clrscr(); printTitle();
    setColor(14);
    printf("\n =====\n"); printf("  PLAYER
    PROFILE SUMMARY:\n"); printf(" Name: %s\n", player.username);
    printf(" Total Words Found: %d\n", player.totalSessionScore); printf(" Puzzles Fully Completed:
    %d\n", player.puzzlesCompleted); printf(" =====\n\n");
    resetColor();

```

3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
27
28
29
30
31
32
33
34
35
36
37
38
39
40
41
42
43
44
45
46
47
48
49
50
51
52
53
54
55
56
57

```
return 0;
```

```
}
```

58

Technical Explanation: The `main()` routine initializes console settings, seeds `srand()`, and collects player username information into `player.username`. It enters a `while(running)` game loop that handles rendering, menu selection, and puzzle regeneration. Upon exiting (Choice 0), the program clears the screen and displays the accumulated statistics stored inside the `PlayerProfile` structure.

4.8 Execution Workflow Summary

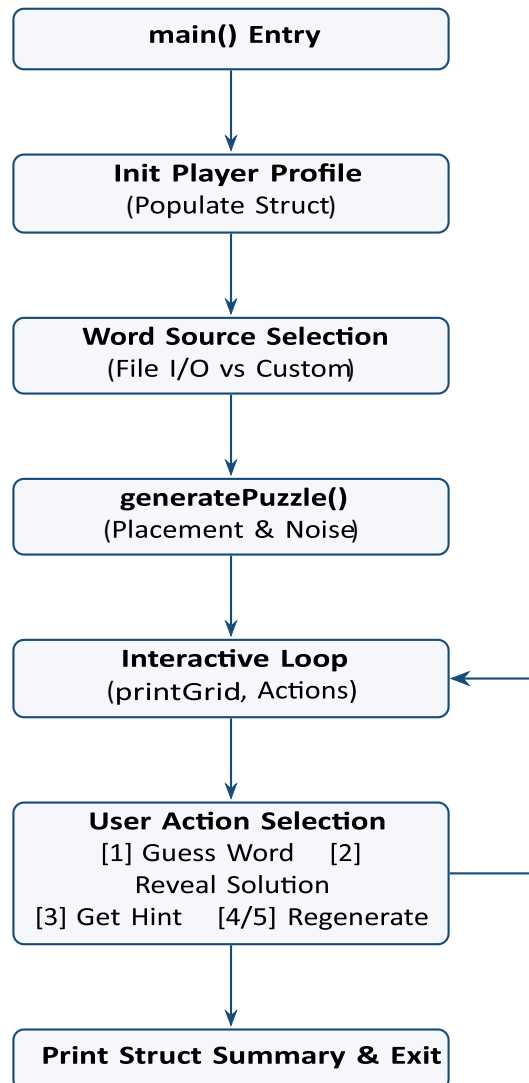


Figure 8: System execution pipeline and interactive state flow.